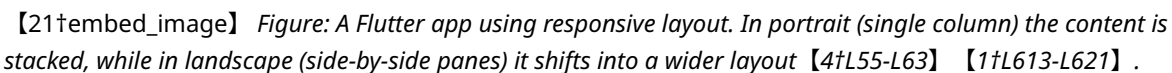


Layout & Safe Area

To prevent content appearing under the status bar or notch, wrap critical screens (e.g. Scaffold bodies) in Flutter's `SafeArea` or manually apply `MediaQuery.padding`. The Flutter docs recommend wrapping your main content in a `SafeArea` so system UI (status bar, notch) won't overlap app content [1†L613-L621] [1†L648-L657]. Additionally, use responsive widgets (e.g. `MediaQuery.orientationOf(context)` or `OrientationBuilder`) to adapt to orientation changes [4†L55-L63] [4†L99-L104]. For example, in portrait mode you might use a single-column layout, while in landscape you could switch to a two-pane layout. This is illustrated below:

 *Figure: A Flutter app using responsive layout. In portrait (single column) the content is stacked, while in landscape (side-by-side panes) it shifts into a wider layout [4†L55-L63] [1†L613-L621].*

By applying these techniques, the UI will automatically inset below the status bar (avoiding overlap) and rearrange when the device rotates. In particular, on phones in landscape you can have a compact two-column view, whereas on tablets (or rotated phones) you might show a sidebar navigation. Using `LayoutBuilder` or `OrientationBuilder` allows you to rebuild the layout based on `MediaQuery` constraints [4†L99-L104]. For instance, you might show the “Design Tree” panel as a drawer on small screens but keep it permanently visible on wider screens. In summary, wrap your Scaffold's body in `SafeArea` and use responsive builders to handle different breakpoints and avoid the UI clipping under the status bar [1†L613-L621] [4†L99-L104].

Android Storage & File Picker

To import fabrics or images, integrate a file-picker plugin and ensure proper Android permissions. Modern Flutter plugins like `file_picker` use Android's **Storage Access Framework (SAF)**, which means the user picks files through a system dialog and your app gains access to those selected files. This does *not* require broad storage permissions [15†L471-L478] [6†L206-L214]. The Android docs explain that on API 19+ a user-chosen document grants temporary access without your app needing `READ_EXTERNAL_STORAGE` [15†L471-L478]. In practice, you simply invoke the picker (e.g. `FilePicker.pickFiles()`); the system UI asks the user, and the plugin returns the file URI. No extra runtime permission is needed for `file_picker` on Android 11+ [6†L206-L214] [15†L471-L478].

However, if you use plugins like `image_picker`, you must declare the appropriate permissions in **AndroidManifest.xml**. For example, on Android 10 and below you may need `READ_EXTERNAL_STORAGE`, and on Android 13+ you should add `READ_MEDIA_IMAGES` (to read photos) or `READ_MEDIA_VIDEO` (for videos) [6†L206-L214] [8†L112-L121]. Be sure to include these entries and request them at runtime if you target older Android versions. In summary, configure your Android manifest with the file/media permissions and use a file picker. The system picker will then allow browsing “Phone Storage” or “Gallery”. If the import dialog currently does nothing, check that the picker plugin is correctly invoked and that any required permissions are in place (especially on older Android targets) [6†L206-L214] [8†L112-L121].

Chat Attachments & DesignState Integration

The AI chat interface should allow attaching images/patterns so that conversation can act on them. In practice, this means extending the chat input to include an “attachment” button that can upload or select files (e.g. motif images, border graphics, color palettes). Conceptually, when a user selects a file or image, you add it to the **DesignState** (or a conversation context) and include a reference in the AI prompt. This approach is common in chat apps: for example, chat SDKs let users upload images and include them as message attachments [18†L146-L155] [18†L182-L191]. In our case, tapping an “Attach” icon should open a picker (gallery, files, or an in-app asset library). The picked file’s URI would then be sent along with the user’s text. Internally, you’d model the conversation as containing both text and attached assets (image, fabric, etc.).

By treating attachments as part of the conversation state, the AI Orchestrator can include them in prompts (e.g. “Given this border image, replace the motif”). Implement a `ChatMessage` model that holds an optional file/asset reference. When a file is attached, dispatch a “SendAttachment” event to your state manager (similar to how chat apps do) [17†L65-L74]. The Orchestrator then recognizes attachments: if the user says “use this motif” after attaching an image, the Constraint Engine will know which region to modify. In short, enable an attachment UI in the chat (a clip icon) and make sure these attachments are fed into the same state object that the AI pipeline reads. This ties the conversation UI to the `DesignState`, so instructions and attached assets together drive the design edits.

Design Tree & Editable Layers

Model the design structure as a hierarchical tree reflecting garment sections and their attributes. For example, the root node might be “Design” with children `Kurta`, `Pant`, and `Dupatta`. Each of these can have sub-nodes: e.g. under `Kurta`: *Base Color, Overall Print, Hero Motif, Border, Sleeve, Neck, Hem*. Under `Dupatta`: *Top Pallu, Middle Field, Bottom Pallu, Side Borders, Corner Motifs*, etc. Each leaf node should have editable properties (color, pattern, motif). For instance, clicking “Top Pallu” could expand options to **repeat pattern**, **add motif**, **change color**, or **import image**. In practice, implement this with nested data classes (using `Freezed`) and ensure each node has flags like `isSelected`, `isVisible`, and its own transformation state (e.g. scale, rotation).

Within each section (e.g. ‘Top Pallu’ of `Dupatta`), the UI can allow dragging in width/height adjustments or swapping motifs. The `DesignTree` schema then drives the canvas layers: only the selected part is modified. There isn’t a formal citation for this specific app design, but this follows common UI patterns where complex objects are edited by selecting subcomponents and exposing relevant tools. In your implementation, define a `DesignTree` or similar class that holds the hierarchy and write UI code to render it (possibly using `ExpansionTile` or a custom tree widget). Ensure zoom/pan gestures are smooth on the tree view so users can comfortably navigate the structure.

Testing & Validation

Finally, write integration tests to verify these flows. For layout, use Flutter’s testing tools to simulate orientation changes. The official docs show how to set `tester.view.physicalSize` for portrait and

landscape and then assert layout differences [【12†L669-L677】](#) . For example, test that in portrait the number of columns is X and in landscape it changes to Y. Similarly, you can simulate tapping the import button and ensure the file picker intent is launched (this might require a mock). For the AI chat, you can write a widget test that enters text and optionally mocks an image attach, then verifies that the chat state contains the expected message and attachment.

By systematically testing each component – safe area adherence, responsive layout, file import, and chat input – you’ll catch any regressions. For instance, the orientation test example from Flutter’s cookbook shows exactly how to assert `MediaQuery.of(context).orientation` and number of columns

[【12†L669-L677】](#) . Mimic this for your design screen (e.g. expect a “Design Tree” panel to appear in landscape). Also test that the import dialog appears (or a permission is requested) when hitting the import icon. End-to-end, you could write a flow: open chat, press “attach”, pick an image (mocked), then send “use this motif”, and verify the DesignState was updated accordingly. This ensures the fixes are robust.

Sources: Flutter’s official docs on **SafeArea/MediaQuery** and responsive design [【1†L613-L621】](#) [【4†L99-L104】](#) , Android’s guide on **Storage Access Framework** [【15†L471-L478】](#) , and community answers on Flutter file pickers [【6†L206-L214】](#) [【8†L112-L121】](#) . These references inform the recommended solutions above.
